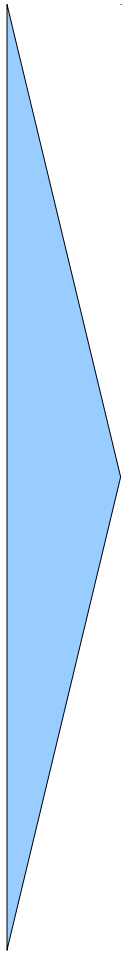
Lecture Overview



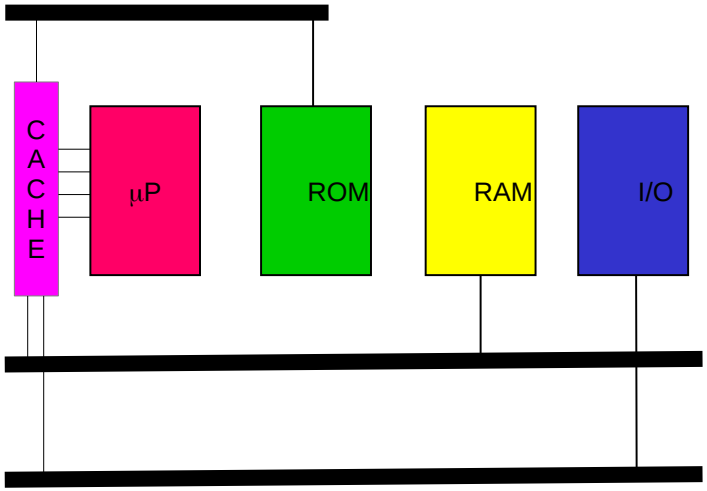
- Introduction, Microcontroller basics
- 32-bit Microcontroller technology
- ARM Cortex M Architecture, (CM0 CM3 CM4 CM7)
- Cortex M Microcontroller Instruction Set
- Exception Handling, Interrupts
- Real-time Operating Systems (RTOS)
- Memory Management, protected memory, MPU
- Implementing Calculations, Data Formats, FPU
- I/O, Device drivers
- Application Design
- Case Studies, Advanced Design Patterns

https://skript.eit.h-da.de/ams/

Memory

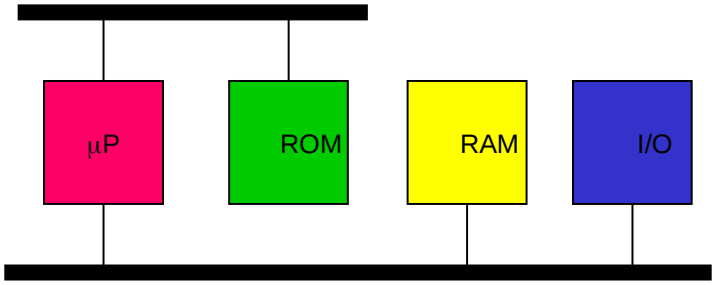
- Memory Architectures (review)
- Performance issues
- Cortex M Memory Architecture
- **Memory Protection Unit**
- Memory Management Unit

Memory Architecture

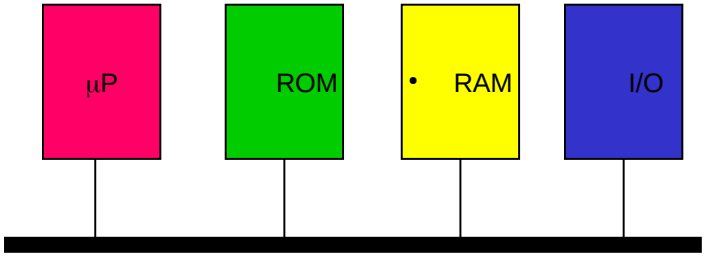


Modified Harvard

Harvard

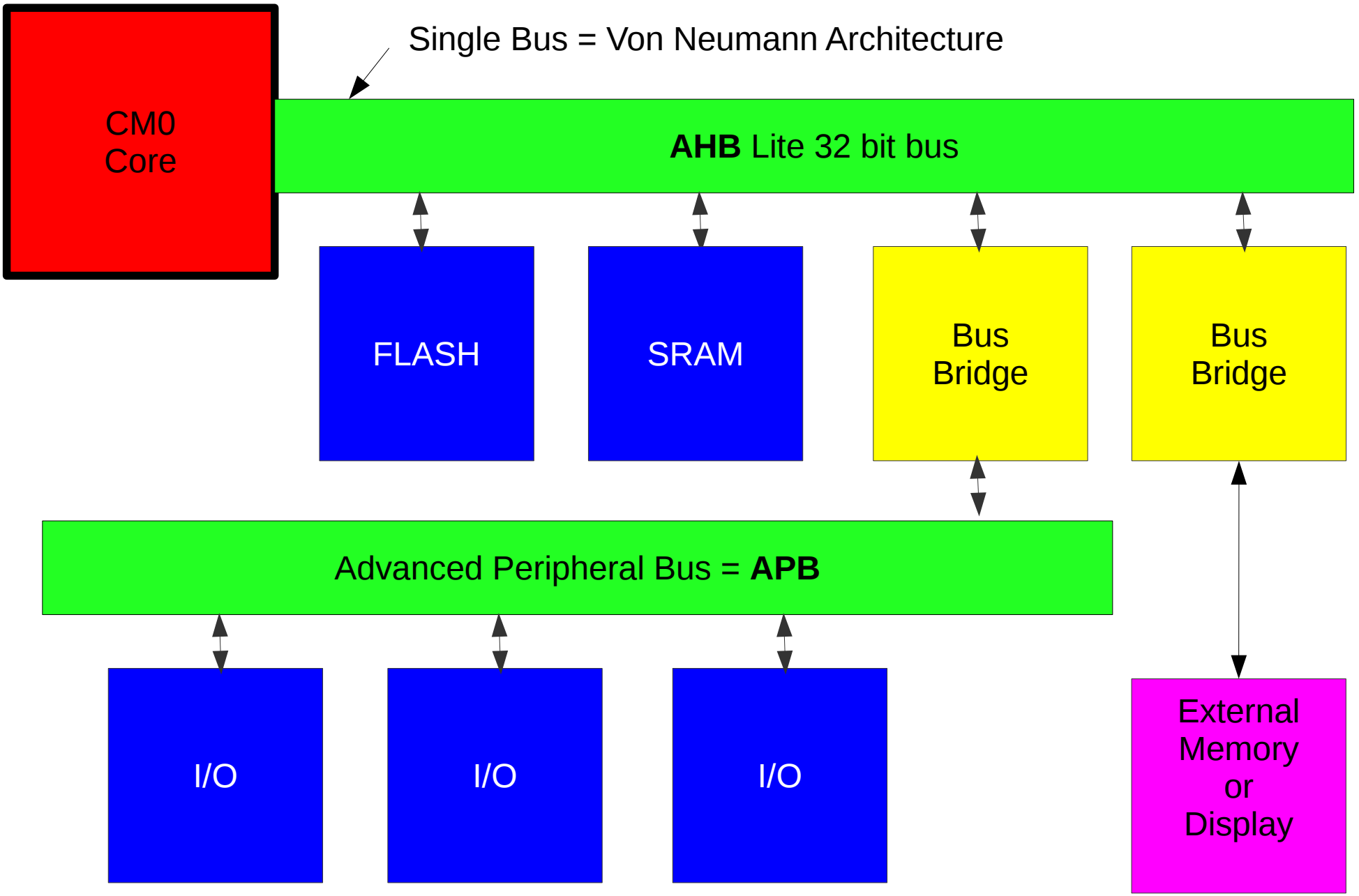


Flat Memory

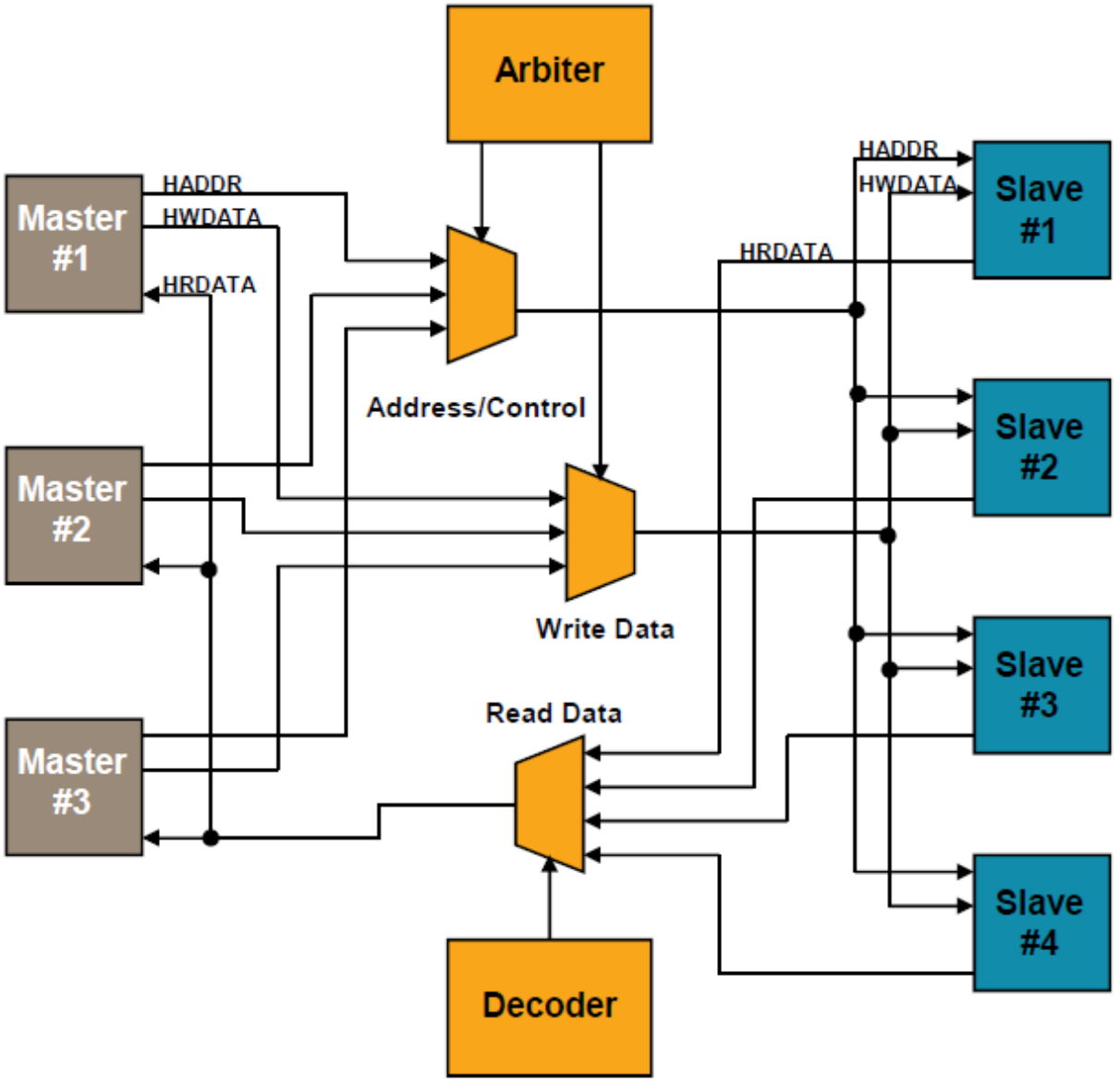


Von Neumann

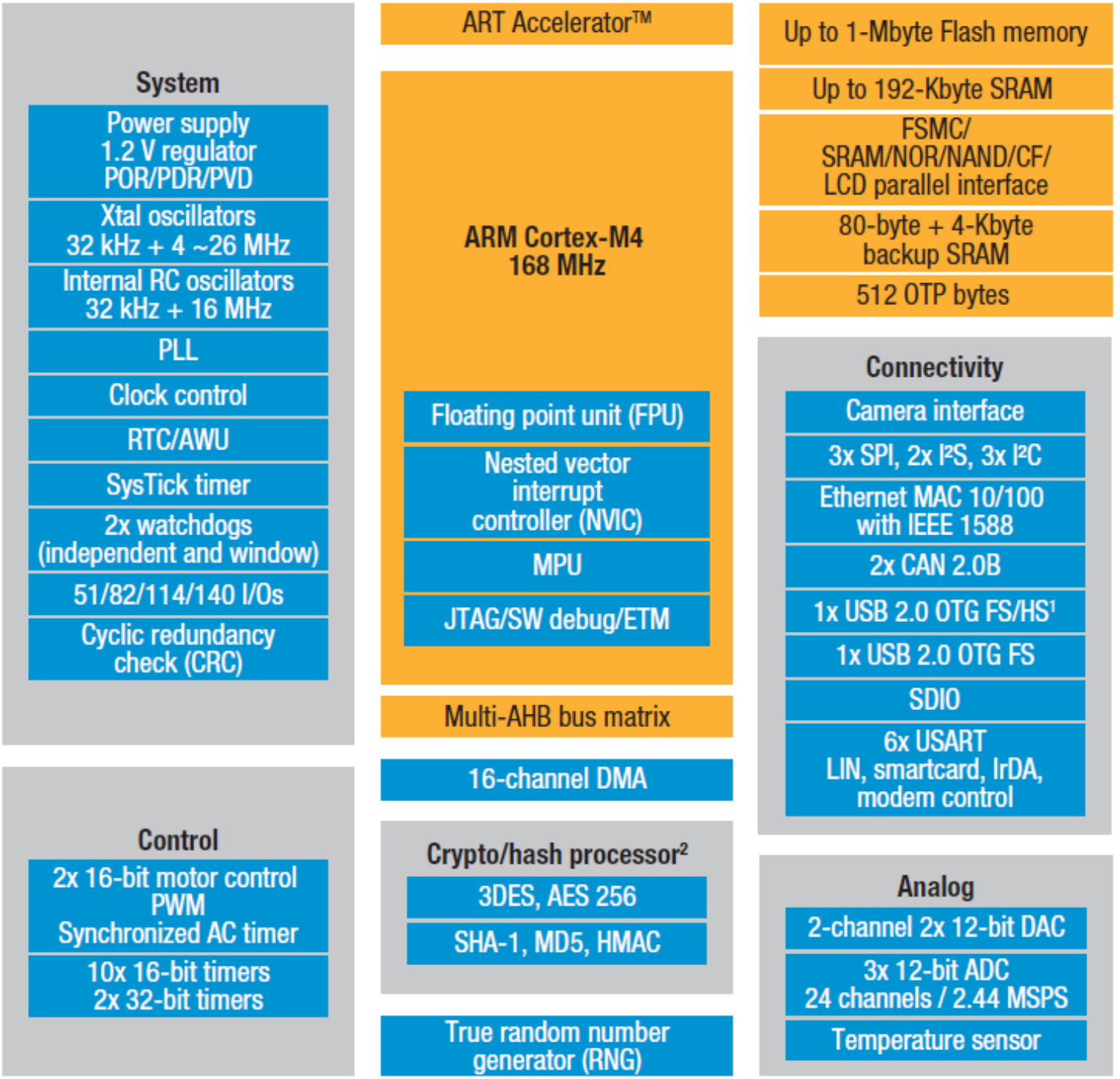
Cortex M0 Bus System



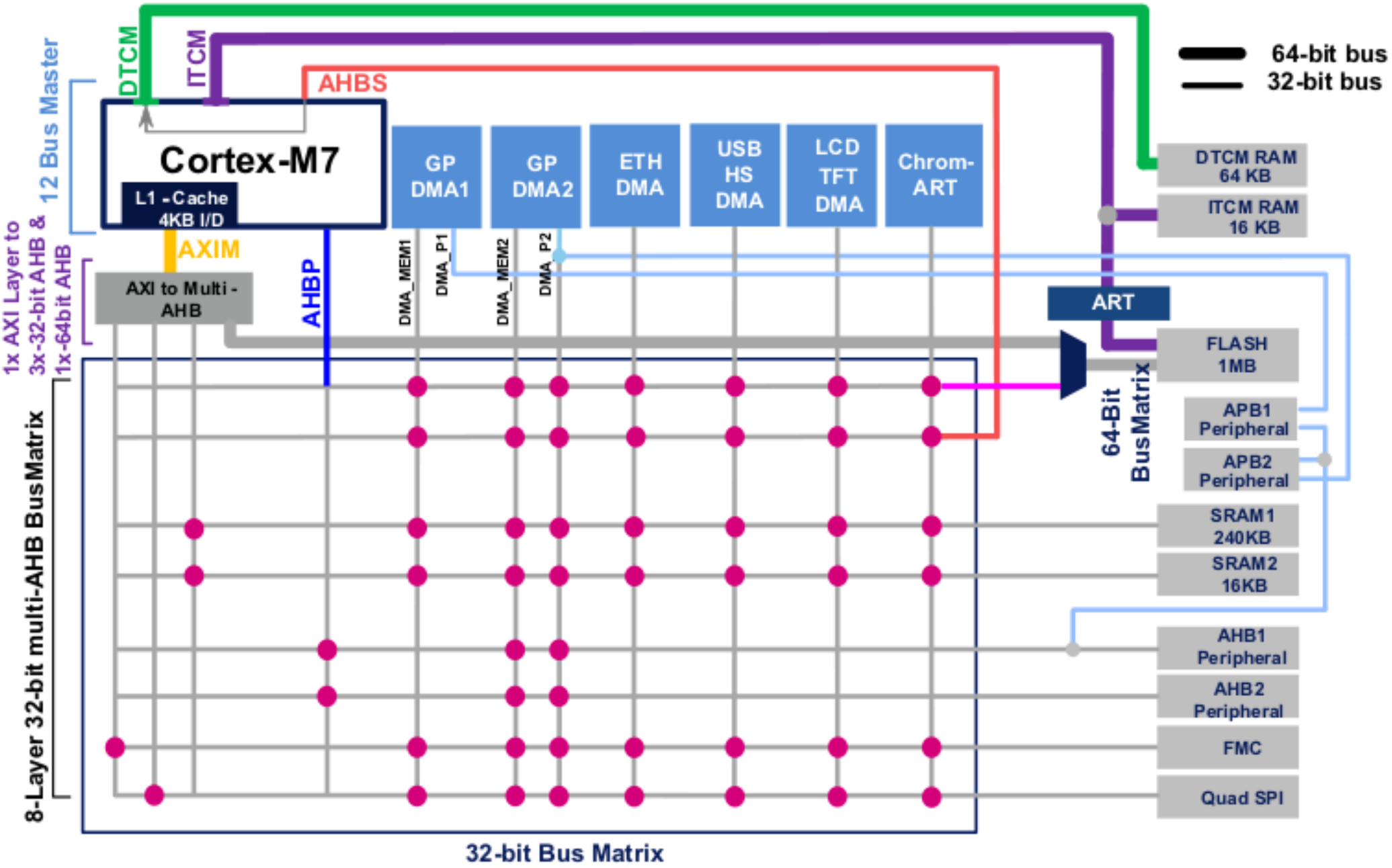
Advanced High-Performance Bus AHB



ARM Cortex M4 SoC

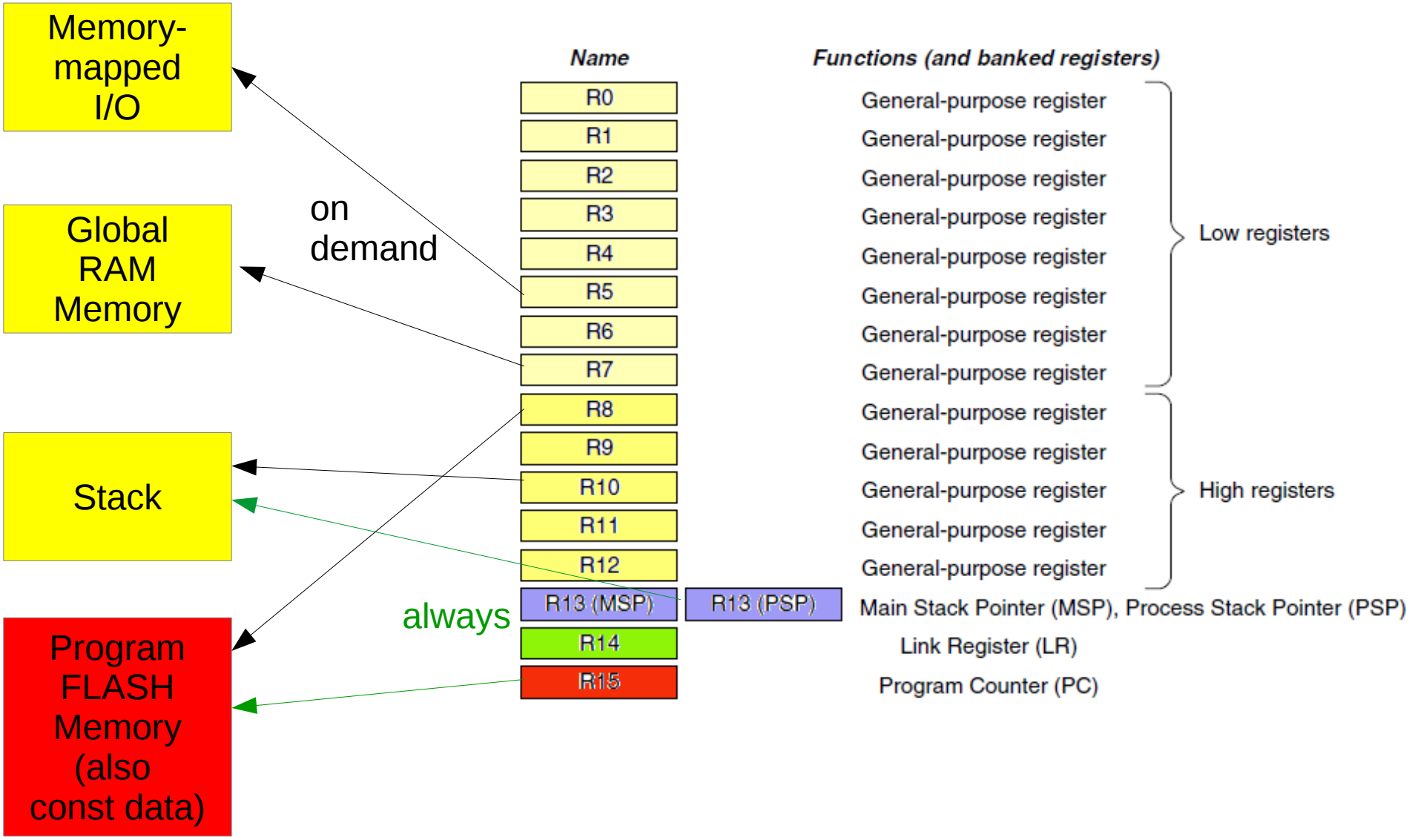


ARM Cortex M7 Bus Systems



Typical Memory Situation

Register pointing to data items



```
int global_variable;
static int global_static_variable;

static const int static_const_int=123;

int sub( int parameter);

extern "C" int mein( int dummy)
{
    int auto_variable=0;
    static int static_local_variable;
    static_local_variable++;

    auto_variable=sub( auto_variable + static_local_variable);
    return auto_variable;
}

int sub( int parameter)
{
    int auto_variable=static_const_int;

    ++global_static_variable;

    auto_variable += parameter;

    int &dynamic_variable=*new int(auto_variable);
    return dynamic_variable;
}
```

Memory Security

At Development Time

- Spotting undesired memory accesses
- Enforcing modularity and information hiding

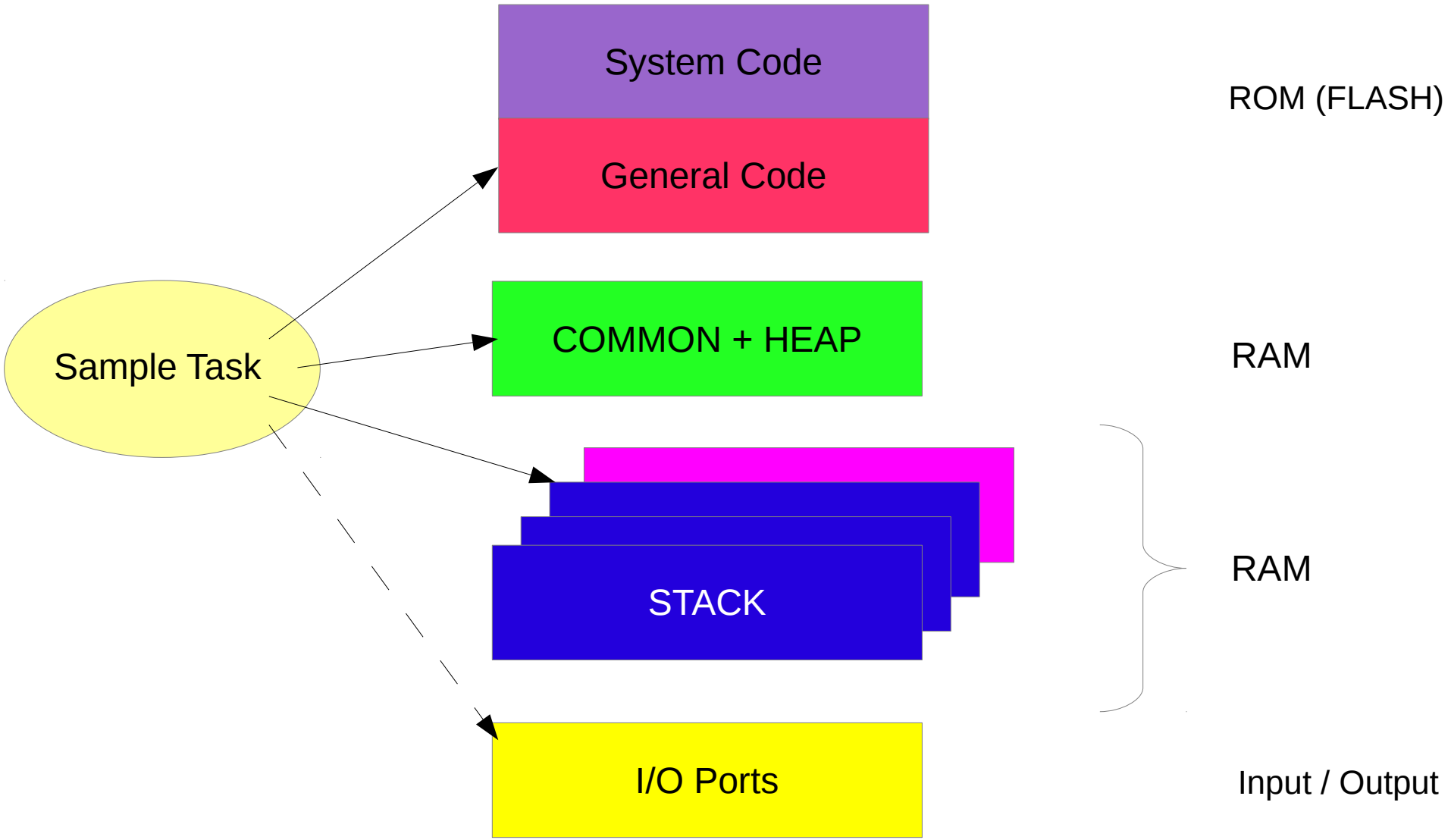
At Runtime

- Security issues
- Robustness of the application
- Instantaneous trapping of errors
- Maintaining software reliability

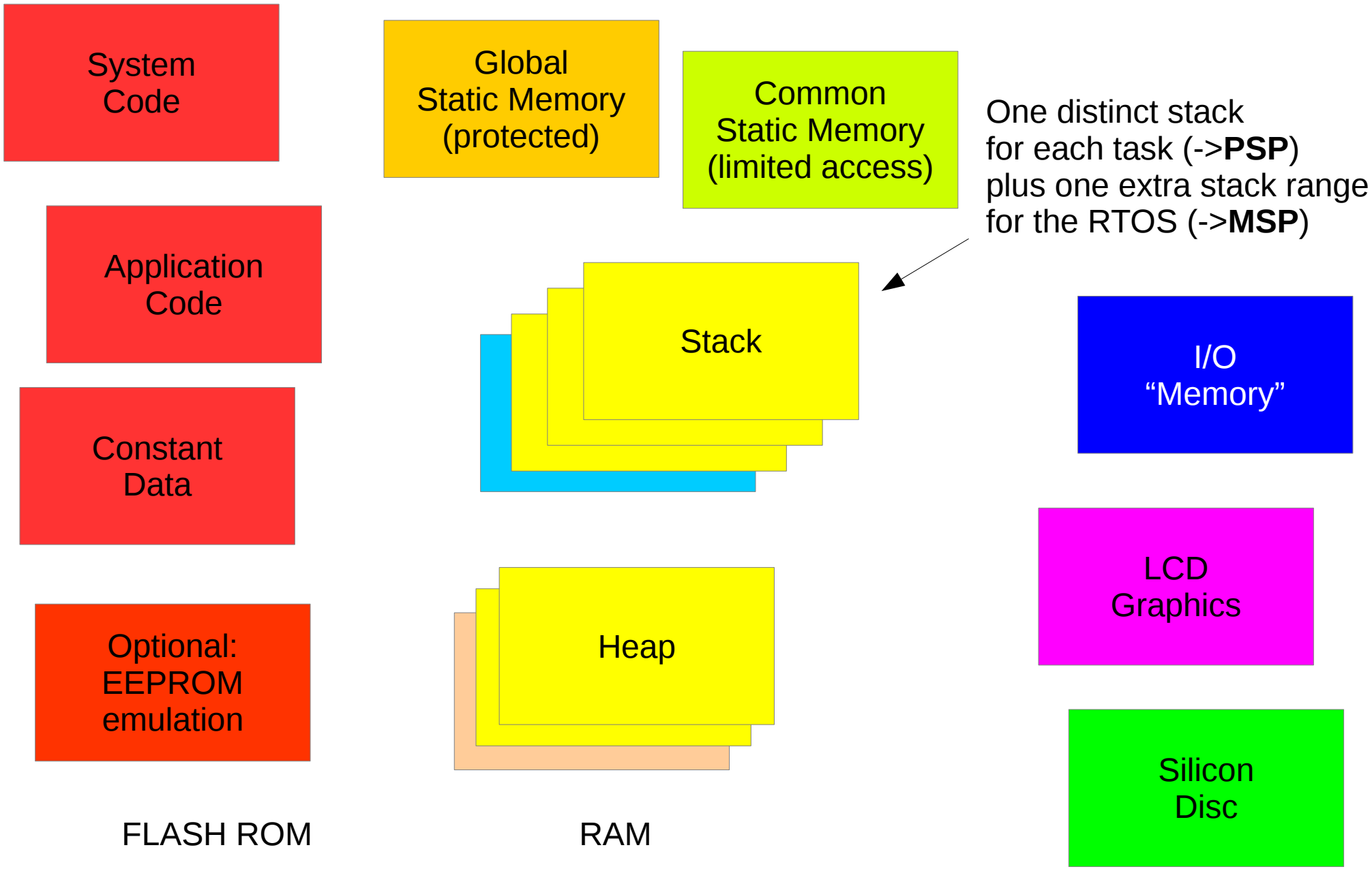
Solution:

- Protected memory architecture

Memory Layout
Logical view



Memory “Types”
Application View



MPU
Memory Protection Unit

Improvement of the reliability and robustness of embedded applications

Making some parts of the memory accessible only by privileged code

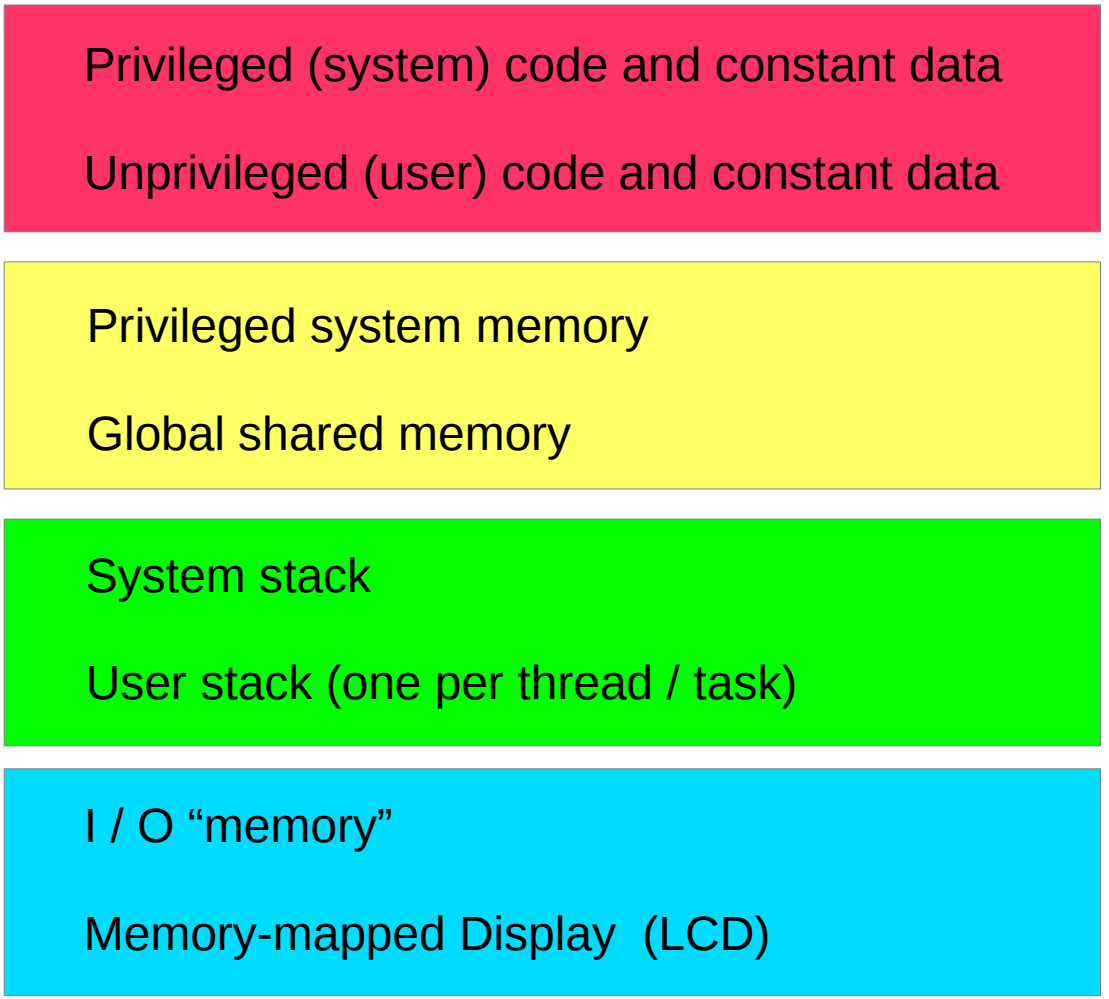
Segmentation of the physical memory into “memory tiles”

Available on some CM0 and CM3 and on most CM4 micro-controllers

Services:

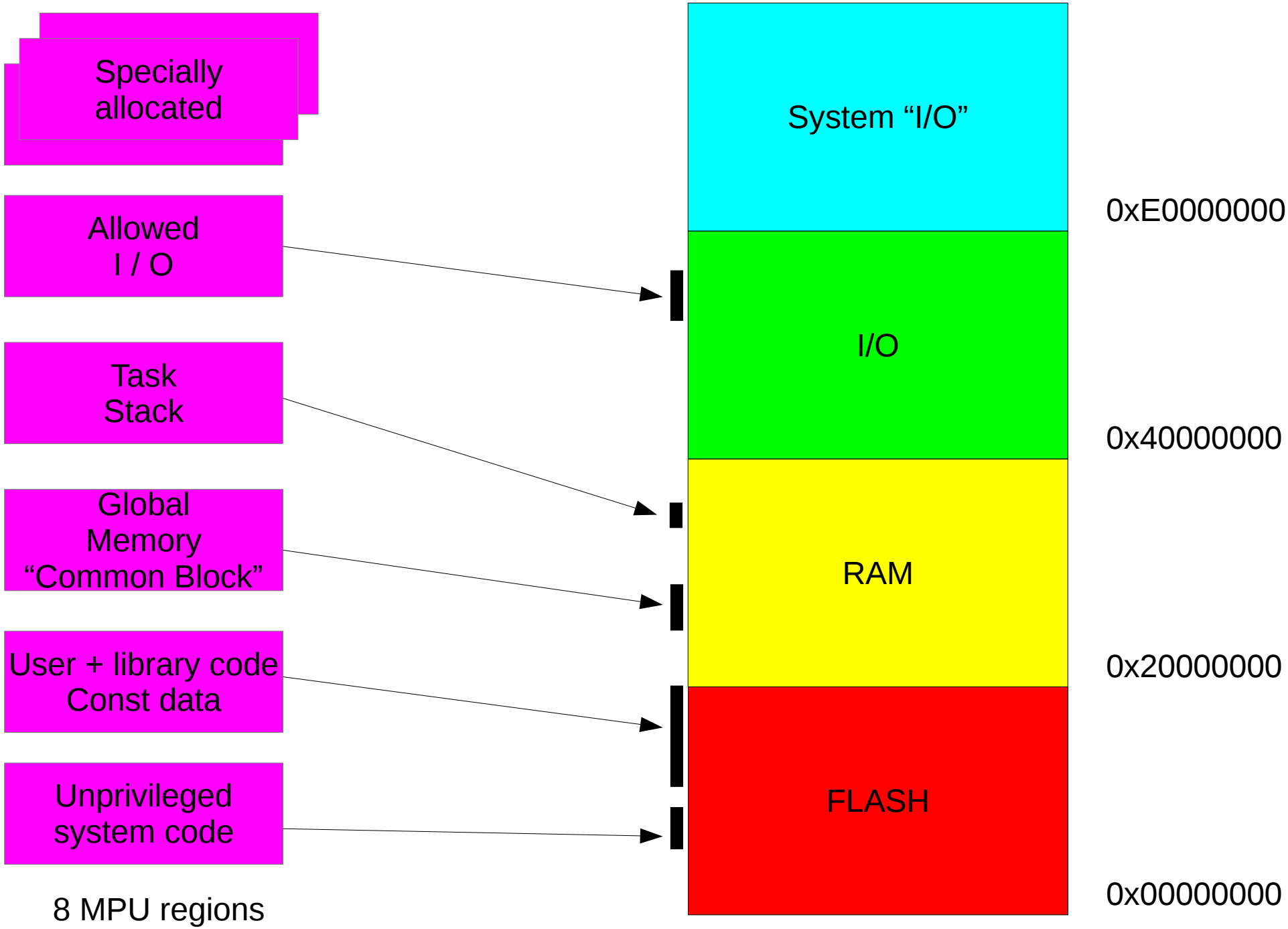
- Preventing user applications from corrupting data used by the operating system
- Separating data between processing tasks
- Protecting vital data regions against unprivileged access
- Disallowing direct I/O / NVIC / system-core access
- Detection of execution errors like stack overflow

MPU regions
Application View

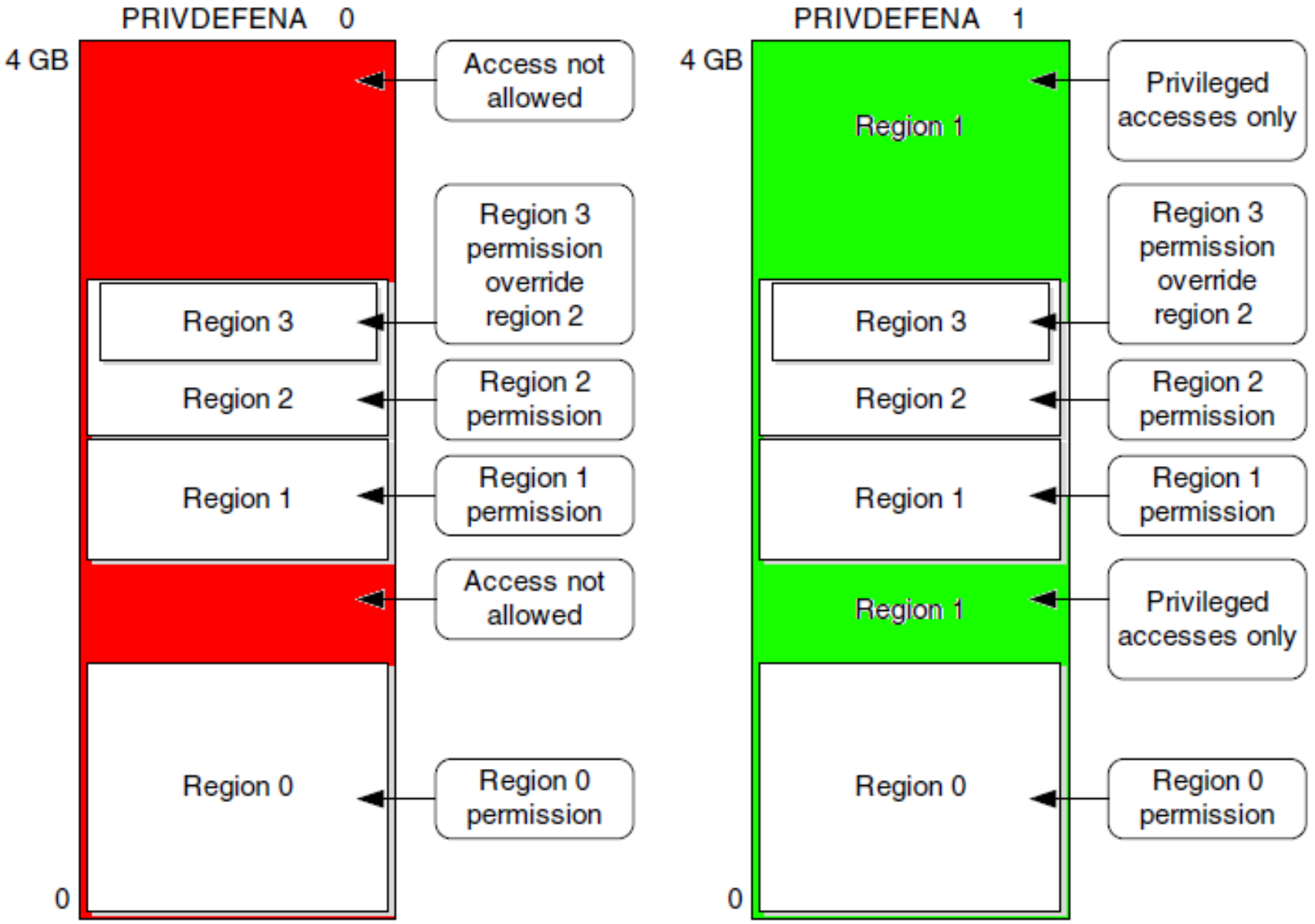


Remark:
Up to eight memory regions are available.
This should be sufficient even for complex systems

Segmented Memory Map



Default Memory Permissions



MPU Registers

MPU region number	Select region to be programmed	(0..7)
MPU region base address	Aligned base address (32 bytes .. 4G bytes alignment) Regions are always aligned. Alignment granularity = region size.	
MPU region attributes+size	Instruction access	XN field
	Data access	AP field
MPU type register	MPU available or not Number of MPU regions	
MPU control register	Enable / Disable the MPU Default memory map background region enabled MPU enabled during hard fault handler	

AP Value	Privileged Access	User Access	Description
000	No access	No access	No access
001	Read/write	No access	Privileged access only
010	Read/write	Read only	Write in a user program generates a fault
011	Read/write	Read/write	Full access
100	Unpredictable	Unpredictable	Unpredictable
101	Read only	No access	Privileged read only
110	Read only	Read only	Read only
111	Read only	Read only	Read only

Example: FreeRTOS MPU support

Fixed Memory Regions:

1) User Code + Libs	(xr unprivileged)
2) System Code	(xr privileged)
3) System Data	(r/w privileged)
4) I/O memory	(r/w unprivileged)
5) Task Stack	(r/w unprivileged)

Distinct for each task

6-8) User-Configurable Memory Regions:

Three more regions for configurable access rights, distinct for every task.

Task Definition
Restricted Tasks

```
/*-----------------------------------------------------------*//**
 * @brief       Task parameters and MPU settings
 *----------------------------------------------------------*/
const xTaskParameters BlinkTaskParameters =
{
    RestrictedBlinkTask,            /* pvTaskCode       */
    (signed char *)"",              /* pcName           */
    BLINK_TASK_STACK_SIZE_WORDS,    /* usStackDepth     */
    ( void * ) (LEDs+2),            /* pvParameters     */
    2 | portPRIVILEGE_BIT,          /* uxPriority       */
    BLINK_Stack,                    /* puxStackBuffer . */
    {                               /* xRegions         */
        /* Base address     Length      Parameters      */
        { (void *)0x40022000, 32,             portMPU_REGION_READ_WRITE},
        { LEDs,               32,             portMPU_REGION_READ_ONLY},
        { 0x00,               0x00,           0x00 }
    }
};
```

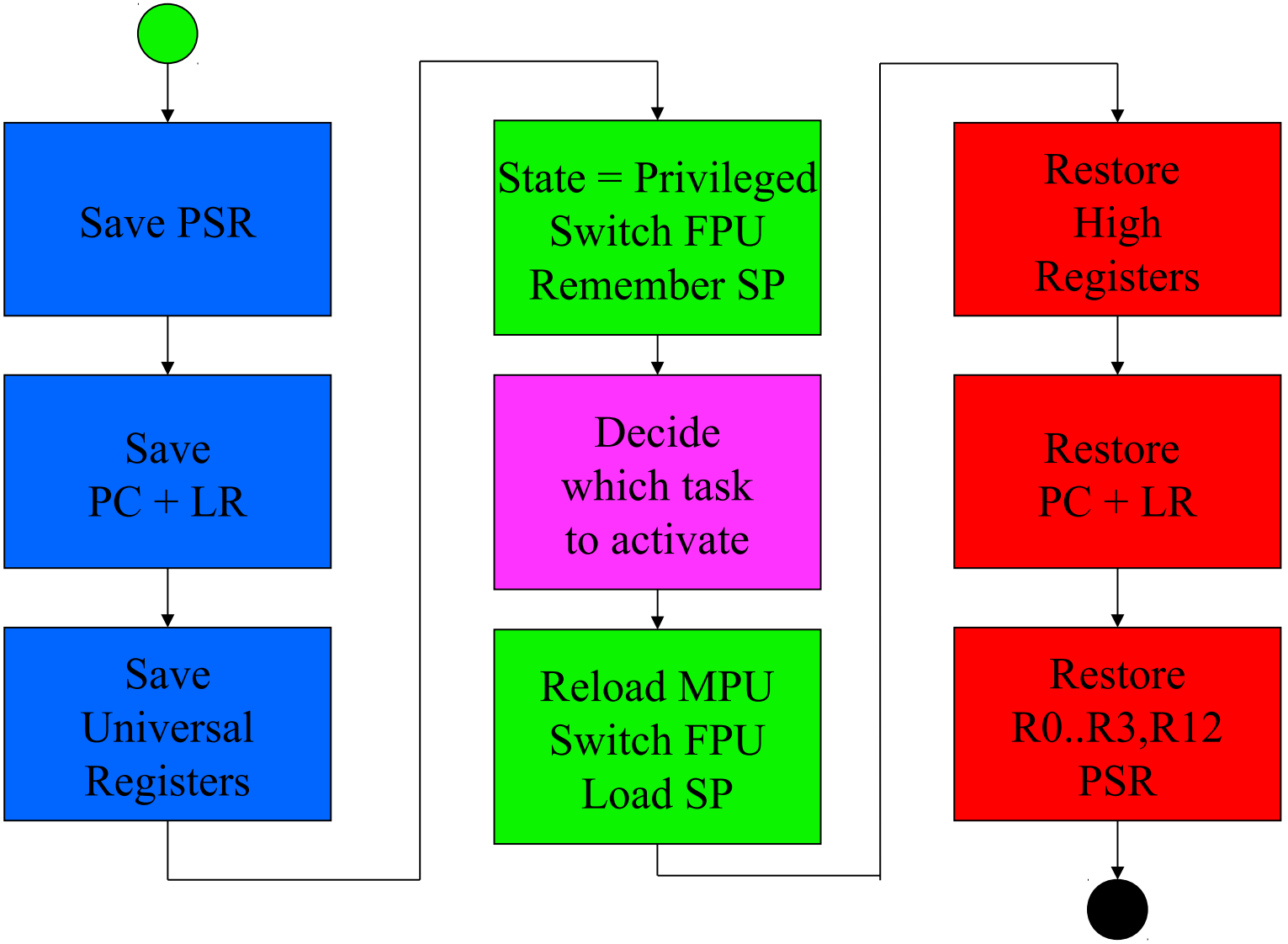
```
xTaskCreateRestricted( &CPP_BlinkTaskParameters,   0 );
```

Task Control Block

```
typedef struct tskTaskControlBlock
{
    volatile portSTACK_TYPE *pxTopOfStack;
    xMPU_SETTINGS           xMPUSettings;
    xListItem               xGenericListItem;
    xListItem               xEventListItem;
    unsigned portBASE_TYPE  uxPriority;
    portSTACK_TYPE          *pxStack;
    signed char             pcTaskName[ configMAX_TASK_NAME_LEN ];
    unsigned portBASE_TYPE  uxBasePriority;
} tskTCB;
```

```
typedef struct MPU_REGION_REGISTERS
{
    unsigned portLONG ulRegionBaseAddress;
    unsigned portLONG ulRegionAttribute;
} xMPU_REGION_REGISTERS;

/* Plus 1 to create space for the stack region. */
typedef struct MPU_SETTINGS
{
    xMPU_REGION_REGISTERS xRegion[ portTOTAL_NUM_REGIONS ];
} xMPU_SETTINGS;
```

Thread Switch / Context Switch



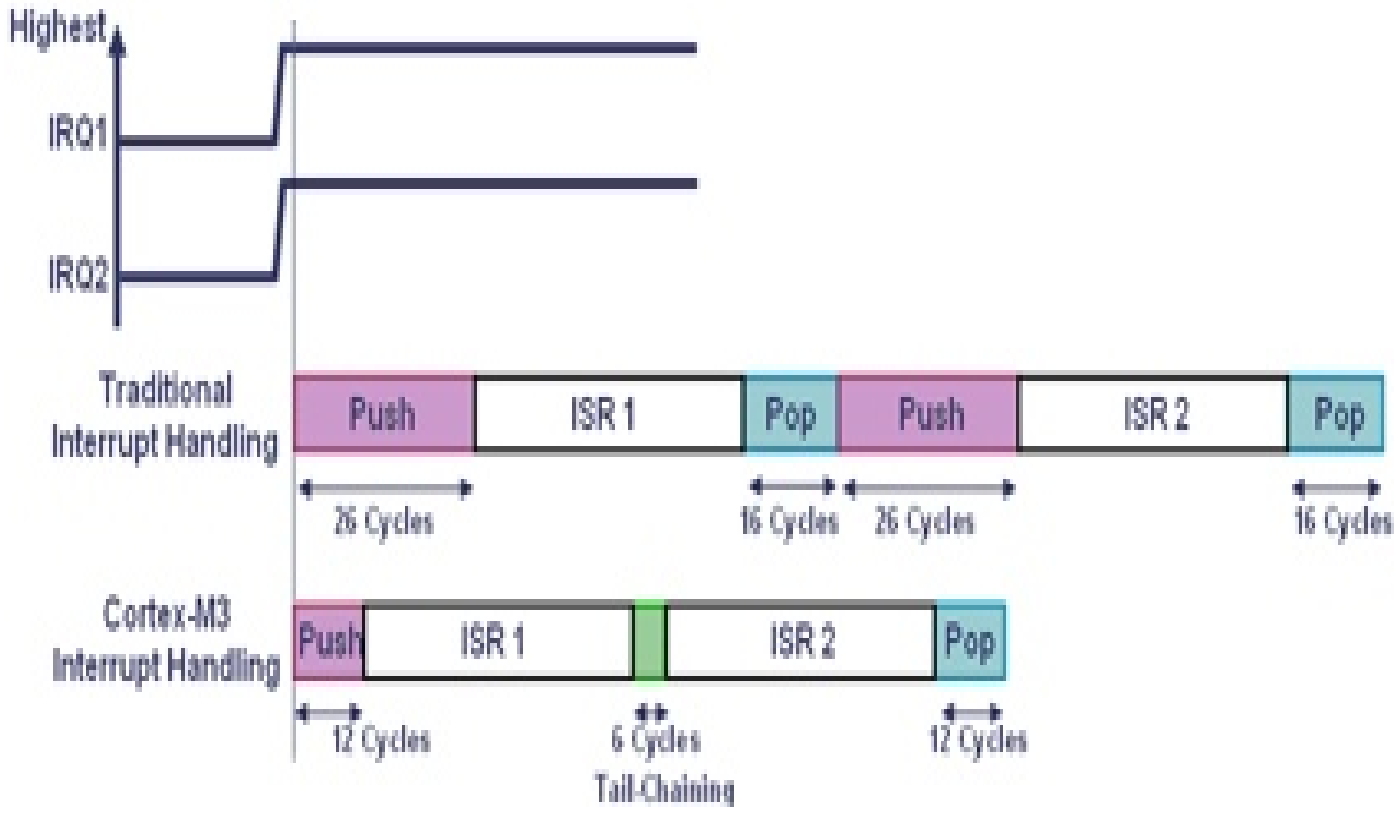
Cortex M Processors do the left and right column automatically by using their SVC / return from interrupt mechanisms.

Preemptive context switch

```
/** @brief  This function handles External line0 interrupt request. */
void EXTI15_10_IRQHandler(void)
{
    portBASE_TYPE TaskWoken = pdFALSE;

    /* reset EXTI I/O interrupt latch */
    EXTI_ClearITPendingBit(KEY_BUTTON_EXTI_LINE);

    /* disable further interrupts for a while */
    NVIC_InitStructure.NVIC_IRQChannelCmd = DISABLE;
    NVIC_Init(&NVIC_InitStructure);

    /* wake up task */
    xQueueSendFromISR( EXTIqueue, 0, &TaskWoken);

    /* trigger timer callback */
    xTimerStartFromISR( de_bounce_Timer, &TaskWoken);

    /* preempt the active task if necessary */
    portEND_SWITCHING_ISR( TaskWoken);
}
```

```
#define portEND_SWITCHING_ISR( xSwitchRequired ) \
        if( xSwitchRequired ) *(portNVIC_INT_CTRL) = portNVIC_PENDSVSET
```

PendSV
will be executed immediately
“tail-chaining”



Context switch with MPU

PendSV handler

```
"    mrs r0, psp                          \n"
"                                         \n"
"    ldr r3, pxCurrentTCBConst            \n" /* Get the location of the current TCB. */
"    ldr r2, [r3]                         \n"
"                                         \n"
"    mrs r1, control                      \n"
"    stmdb r0!, {r1, r4-r11}              \n" /* Save the remaining registers. */
"    str r0, [r2]                         \n" /* Save the new top of stack into the first member of the TCB. */
"                                         \n"
"    stmdb sp!, {r3, r14}                 \n"
"    mov r0, %0                           \n"
"    msr basepri, r0                      \n"
"    bl vTaskSwitchContext                \n"
"    mov r0, #0                           \n"
"    msr basepri, r0                      \n"
"    ldmia sp!, {r3, r14}                 \n"
"                                         \n" /* Restore the context. */
"    ldr r1, [r3]                         \n"
"    ldr r0, [r1]                         \n" /* The first item in the TCB is the task top of stack. */
"    add r1, r1, #4                       \n" /* Move onto the second item in the TCB... */
"    ldr r2, =0xe000ed9c                  \n" /* Region Base Address register. */
"    ldmia r1!, {r4-r11}                  \n" /* Read 4 sets of MPU registers. */
"    stmia r2!, {r4-r11}                  \n" /* Write 4 sets of MPU registers. */
"    ldmia r0!, {r3, r4-r11}              \n" /* Pop the registers that are not automatically saved on exception entry. */
"    msr control, r3                      \n"
"                                         \n"
"    msr psp, r0                          \n"
"    bx r14                               \n"
```

“Order” PendSV : *(portNVIC_INT_CTRL) = portNVIC_PENDSVSET;

TaskSwitchContext

```
void vTaskSwitchContext( void )
{
    if( uxSchedulerSuspended != ( unsigned portBASE_TYPE ) pdFALSE )
    {
        /* The scheduler is currently suspended - do not allow a context
        switch. */
        xMissedYield = pdTRUE;
    }
    else
    {
        /* Find the highest priority queue that contains ready tasks. */
        while( listLIST_IS_EMPTY( &( pxReadyTasksLists[ uxTopReadyPriority ] ) ) )
        {
            configASSERT( uxTopReadyPriority );
            --uxTopReadyPriority;
        }

        /* listGET_OWNER_OF_NEXT_ENTRY walks through the list, so the tasks of the
        same priority get an equal share of the processor time. */
        listGET_OWNER_OF_NEXT_ENTRY( pxCurrentTCB, &( pxReadyTasksLists[ uxTopReadyPriority ] ) );
    }
}
```

ROM and COMMON

Macros to force data placement in ROM/FLASH or in a special RAM area

(Header File **memory.h)**

COMMON:

```
#define COMMON __attribute__ ((section ("common_data")))
```

ROM:

```
#define ROM const __attribute__ ((section (".rodata")))
```

```
CONSTEXPR_ROM (C++ special):

#define CONSTEXPR_ROM constexpr __attribute__ ((section (".rodata")))
```

Volatile Memory

- I/O Memory
- Memory accessible by other processors
- Memory used by ISR’s and/or other tasks
- Keyword **volatile** mandatory
  Compiler will not optimize the code assuming the information is persistent
- LDM / STM Instructions may be ***postponed*** by an interrupt
  and repeated after completion of the ISR !
- Some I/O cells require byte or halfword access !
- Compiler optimization may be dangerous !
  Hard-to-find errors could come up !

Outlook:
Memory Management Unit

Address translation physical → logical

Establishing “Processes” with virtual memory

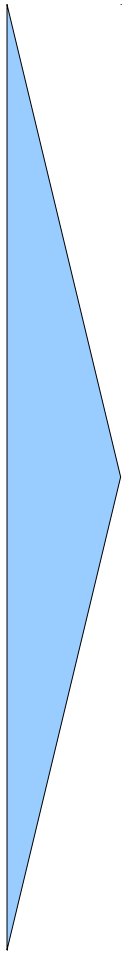
Memory protection support

Memory swapping support

Virtual processing support

Used in application processors and complex embedded systems

Lecture Overview



- Introduction, Microcontroller basics
- 32-bit Microcontroller technology
- ARM Cortex M Architecture, (CM0 CM3 CM4 CM7)
- Cortex M Microcontroller Instruction Set
- Exception Handling, Interrupts
- Real-time Operating Systems (RTOS)
- Memory Management, protected memory, MPU
- Implementing Calculations, Data Formats, FPU
- I/O, Device drivers
- Application Design
- Case Studies, Advanced Design Patterns

https://skript.eit.h-da.de/ams/